NCSM301 · PROBLEM STATEMENT P1

Campus Lost & Found Portal

A PHP, MariaDB and jQuery web application

Project Report, IIT (ISM) Dhanbad

Solution to Problem Statement P1(Campus Lost-and-Found Portal): requirements, design, implementation, security notes, testing and limitations

Team (Roll Nos.): 24JE0002, 24JE0012, 24JE0048, 24JE0449

Project summary

This project is a Campus Lost & Found portal built for problem statement P1. It uses HTML, CSS, JavaScript, jQuery and PHP, with MariaDB/MySQL as the database and Apache as the web server. Users can report lost or found items, search and filter the reports, submit claims and resolve reports. Each report has one of three statuses: “Open”, “Claim Requested” or “Resolved”. New reports are sent with a normal form post, while claims and resolution and view claims use jQuery AJAX. To resolve a report, the person has to enter the email address stored on that report, and `status.php` checks it on the server. The code uses PHP with PDO (PHP data objects) and a two-table schema (`reports` and `claims`). There are no user accounts or sessions.

Abstract

This report describes our solution to problem statement P1, the Campus Lost & Found Portal. People can report lost or found items with a type, title, category, location, date and description, search and filter the reports, submit claims, and follow the status of each report (Open, Claim Requested or Resolved). Resolving a report updates the page without a full reload, which meets the requirement that at least one operation updates without a reload.

The project has three parts. `index.php`, `style.css` and jQuery make up the page that shows the forms, the report list and the search filters, and that sends the claim and resolve requests. Three small PHP files, `add.php`, `claim.php` and `status.php`, receive the data and update the database through PDO. `database.sql` defines two tables, `reports` and `claims`. The one step that needs a check is resolving a report: `status.php` verifies the report ID, the requested status, the current status and the reporter email before it changes anything. The project is deliberately small. It has no user accounts, sessions, real email verification or administrator controls, and section 7 lists these limits.

1. Problem Statement and Requirements

Build a portal to report lost/found items with category, location, date and description. Users must search/filter reports, submit a claim, and track status such as Open, Claim Requested and Resolved; at least one listing/status operation should update without a full page reload.

Requirement compliance

Requirement	How it is met	Where
Report lost/found items	Report form with a Lost/Found selector; new reports start as Open (the default of the status column)	`index.php`, `add.php`, `database.sql`
Category, location, date, description	Form fields saved in category, location, date_occurred and description	`add.php`, `database.sql`
Search reports	Keyword matched against title, description and location	`index.php`
Filter reports	Filters by type, category and status; the Clear link resets them	`index.php`
Submit a claim	Claim form (name, email, explanation) saved in the claims table	`index.php`, `claim.php`
Track status	Open, Claim Requested and Resolved, stored in reports.status	`claim.php`, `status.php`, `database.sql`
Update without full reload	Resolve posts to status.php with jQuery AJAX and the status cell changes in place	`index.php`, `status.php`
Allowed stack only	HTML, CSS, JavaScript, jQuery, PHP, MariaDB/MySQL, Apache	All files

Claims are also sent with AJAX, but the page reloads afterwards so that the new status shows. The no-reload requirement is therefore met by the **Resolve** and the **View Claims** operation.

2. Technology and Data Model

2.1 Technology stack

jQuery 3.7.1 is loaded from the code.jquery.com CDN in `index.php`, so the claim and resolve features need internet access unless a local copy of jQuery is used.

Item	Specification
Frontend	HTML, CSS, JavaScript, jQuery 3.7.1
Backend	PHP with PDO and the PDO MySQL driver (the README asks for PHP 7.4 or newer)
Database	MariaDB, database name `lost_and_found`
Web server	Apache httpd through XAMPP
Database access	PDO prepared statements. `db.php` connects to host 127.0.0.1 as user `lnf_user` with charset utf8mb4, and sets exception error mode.

2.2 Data model

The `reports` table holds one row per item and the `claims` table holds one row per claim. `claims.report_id` is a foreign key with ON DELETE CASCADE, so deleting a report would also delete its claims. `database.sql` also

inserts six sample reports (four Open, one Claim Requested and one Resolved). Importing it more than once can create duplicate sample rows.

Table / column	Purpose
reports.id	Unique ID of the report (auto-increment)
reports.type	lost or found
title, category, location	Short title, item category, and where the item was lost or found
date_occurred, description	Date of the incident and item details
reporter_name, reporter_email	Who made the report; the email is used later to confirm resolution
reports.status	Open, Claim Requested or Resolved (default Open)
created_at, updated_at	Creation time and last update time
image_url	Exists in the schema but is not used by any code
claims columns	id, report_id, claimer_name, claimer_email, message, created_at

3. System Architecture

The page shows the forms and the report list and collects the input. The PHP files receive that input and make the database changes.

Part	Files	Responsibility	Notes
Presentation	`index.php`, `style.css`	Forms, report list, search filters, jQuery AJAX code	Reads filters from the URL
Create	`add.php`	Inserts a report	Redirects to index.php afterwards
Claim	`claim.php`	Inserts a claim, updates status	Open to Claim Requested
Resolve	`status.php`	Validates the request and resolves the report	Checks the reporter email on the server
Connection	`db.php`	Creates the PDO connection	Credentials are in one file
Storage	`database.sql`	Tables and sample data	Foreign key with cascade

3.1 Status lifecycle

```
Open  →  Claim(s) Requested→  Resolved
```

A new report starts as Open. A claim on an Open report changes it to Claim Requested. Entering the matching reporter email on a Claim Requested report changes it to Resolved. A claim made on a report that is already Claim Requested is still saved in the claims table, but the status stays as it is.

4. Implementation

4.1 Submitting a report

The **Submit a Report** form is a normal HTML `POST` form (flow: `index.php → add.php → reports`). It asks for type, title, category, location, date, reporter name, reporter email and description. The category list is fixed in `index.php`: Electronics, Documents & ID, Bags & Luggage, Books & Notes, Accessories, Personal Items, Keys, Clothing and Other. Every field is marked required in the form, and `add.php` checks again that none of them is empty. It then inserts the row with a prepared statement. The status is not set in the query, so the database

default, Open, applies. Finally `add.php` redirects back to `index.php`. If a field is empty or the insert fails, it prints a short error message with a Go back link instead.

4.2 Searching and filtering

`index.php` reads the search and filter values from the URL (GET). The search word is matched with LIKE against title, description and location. The list can also be filtered by type (Lost or Found), category and status, and the filters can be combined. Reports are shown newest first, and the heading shows how many reports matched. The table shows type, title, category, location, date, description, reporter name, status and actions; the reporter's email is not displayed. After a search, the search box and the three drop-downs keep the values that were chosen. **Clear** returns to the full list.

4.3 Submitting a claim (AJAX)

A **Claim** button appears on every report that is not Resolved. Clicking it opens a pop-up form for name, email and an explanation. jQuery sends the data to `claim.php` without leaving the page (flow: `index.php` JavaScript → `claim.php` → claims table and `reports.status`). `claim.php` checks that the report ID and the three fields are present, saves the claim, and changes the report to Claim Requested if it is currently Open. The report ID is kept in a hidden field of the form, and Cancel closes the pop-up without sending anything. If a field is missing or the insert fails, `claim.php` replies with a short error message (All fields are required. or Could not submit claim.). Whatever the reply is, it is shown in an alert, the pop-up closes and the page reloads so that the current status is visible.

4.4 Resolving a report (AJAX)

A **Resolve** button appears on reports whose status is Claim Requested. Clicking it opens a browser prompt for the reporter's email, and then the report ID, the status Resolved and the email are posted to `status.php`. If the prompt is cancelled or left empty, nothing is sent. `status.php` updates the row only when the report is currently Claim Requested and the email matches `reporter_email`. The comparison is done in SQL with `LOWER(TRIM(...))`, so letter case and surrounding spaces do not matter. When the update works, the status cell on the page changes to Resolved and the Resolve button is replaced by a dash, without a reload. The page changes the cell only when the reply contains the word "updated" (the success message is Status updated to Resolved.). Every reply is shown in an alert. With a wrong email the report stays unchanged and the message says the email does not match; the same message appears if the report is not Claim Requested.

5. Resolution Check and Security Notes

Because there are no accounts or sessions, the email check in `status.php` is the only protection on resolving a report. The table lists what the code checks and where.

Risk	Control	Where
Invalid report ID	Request is rejected if the ID is not a positive number	`status.php`
Unexpected status value	Request is rejected if the status is not Open, Claim Requested or Resolved	`status.php`
Missing email	Resolving is rejected if no email is sent	`status.php`
Resolving too early	The update only applies to a report that is currently Claim Requested	`status.php`
Wrong or mistyped email	Email must match reporter_email after LOWER(TRIM(...))	`status.php`
SQL injection	All queries use PDO prepared statements	`index.php`, `add.php`, `claim.php`, `status.php`
Script in displayed text	Report fields and statuses are printed through htmlspecialchars	`index.php`

5.1 Residual risk

The email check only shows that the person knows the email address stored on the report, so anyone who knows or guesses it could resolve that report. It is **not strong authentication**, which is acceptable for a classroom project. Also, `status.php` accepts the values Open and Claim Requested without any email check. The page never sends them, but a request sent directly to the file would be accepted. The database user name and password are written in plain text in `db.php`.

6. Installation

Linux (Fedora/RHEL, using dnf): install httpd, php, php-mysqlnd and mariadb-server, start httpd and mariadb, create the `lost_and_found` database and the user `lnf_user` (for localhost and 127.0.0.1), import `database.sql`, copy the project to /var/www/html/lostfound, and check the settings in `db.php`.

Windows (XAMPP): install XAMPP in C:\xampp, copy the project to C:\xampp\htdocs\lostfound, start Apache and MySQL, create the database and user in phpMyAdmin, import `database.sql`, and check `db.php`. In both cases the application opens at http://localhost/lostfound/index.php.

7. Conclusion

The portal covers the full cycle of reporting, searching, claiming and resolving an item, using only the stack the problem statement allows. Resolving a report is checked on the server against the reporter's email. That check is also the main weakness, because it is not real authentication and would need to be replaced before any real use.